

PROCESS SCHEDULER ALGORITHMS IMPLEMENTATION REPORT

Supervisor by : DR. Rania Ziedan

Team Members

S#	Student Name	Student ID
1	اية محمد سيد	4
2	حنين مصطفى محمد عطا مصطفى	8
3	نادية كامل محمد علي	36
4	هاجر علي الرفاعي علي محمد	37
5	هنا احمد نبهان عودة	38

Table of Contents

1. Introduction

2. Algorithms Used

- Theoretical Background
- Advantage and Disadvantage
- Real Applications and Use case

3. Implementation Details Section

- PCB Structure
- Overall Logic
- Snippets of Implementation parts in code

4. Test Cases output

- Evaluation and Impact on Performance of System

5. Challenges Faces and it's Solution

6. Comparison of Each Algorithms by evaluation matrix

7. Conclusion

1. Introduction

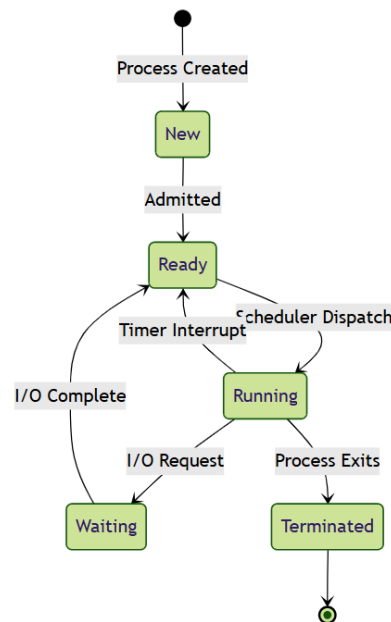
Process scheduling is one of the most critical components of an operating system that determines how the CPU time is allocated among competing processes. It directly impacts **system performance**, response time, and resource utilization. Understanding process scheduling algorithms is essential for system administrators, developers, and anyone working with operating systems

What is Process Scheduling?

Process scheduling is the mechanism by which an operating **system decides which process should be executed by the CPU at any given time**. When multiple processes compete for CPU resources, the scheduler must make decisions to **optimize system performance while ensuring fairness and meeting various quality** of service requirements.

The Process States and Scheduling Context

Before diving into scheduling algorithms, it's crucial to understand process states and how they relate to scheduling decisions.



The scheduler primarily works with processes in the **Ready** state, selecting which process should transition to the **Running** state. This decision-making process is where different scheduling algorithms come into play.

CPU scheduling decisions may take place when a process:

1. Switches from running to waiting state
2. Switches from running to ready state
3. Switches from waiting to ready
4. Terminates

For situations Switches from running to waiting state and Terminates State, there is no choice in terms of scheduling as if a new process (if one exists in the ready queue) must be selected for execution.

For situations Switches from running to ready state and Switches from waiting to ready States, however, there is a choice.

The Preemptive and Nonpreemptive Scheduling

Under Nonpreemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases it either by terminating or by switching to the waiting state and then preemptive is vice versa

The primary objectives of process scheduling include

CPU Utilization: Maximize the percentage of time the CPU is busy executing processes

Throughput: Maximize the number of processes completed per unit time

Turnaround Time: Minimize the total time from process submission to completion

Waiting Time: Minimize the time processes spend waiting in the ready queue

Response Time: Minimize the time from request submission to first response

Fairness: Ensure all processes get reasonable access to CPU resources

The Different Process Scheduling Algorithms

- First- Come, First-Served (FCFS) Scheduling
- Shortest-Job-First (SJF) Scheduling
- Shortest Remaining Time First Scheduling (SRT)
- Round Robin (RR)
- Priority Scheduling
- Multilevel Queue
- Multilevel Feedback Queue

3.Implementation Details Section

General PCB Structure

```
public class Process {  
    //Basic Info  
    public String pid;  
    public int arrivalTime;  
    public int burstTime;  
    // Execution Tracking  
    public int startTime = -1;  
    public int remainingTime;  
    public int completionTime;  
    // Performance Metrics  
    public int waitingTime;  
    public int turnaroundTime;  
    public int responseTime;  
    // Priority - Aging Solution  
    public int basePriority;  
    public int currentPriority;  
    public int waitingTimeCounter;  
  
    public Process(String pid, int arrivalTime, int burstTime, int priority) {  
        this.pid = pid;  
        this.arrivalTime = arrivalTime;  
        this.burstTime = burstTime;  
        this.remainingTime = burstTime;  
        this.basePriority = priority;  
        this.currentPriority = priority;  
        this.waitingTimeCounter = 0;  
    }  
}
```

General Testcase Input

```
//General TestCase
public static List<Process> getGeneralTestCase() {
    List<Process> processes = new ArrayList<>();
    processes.add(new Process(pid: "P1", arrivalTime: 0, burstTime: 6, priority: 1));
    processes.add(new Process(pid: "P2", arrivalTime: 1, burstTime: 4, priority: 2));
    processes.add(new Process(pid: "P3", arrivalTime: 2, burstTime: 7, priority: 3));
    processes.add(new Process(pid: "P4", arrivalTime: 3, burstTime: 1, priority: 4));
    processes.add(new Process(pid: "P5", arrivalTime: 5, burstTime: 4, priority: 4));
    processes.add(new Process(pid: "P6", arrivalTime: 6, burstTime: 7, priority: 3));
    return processes;
}
```

First- Come, First-Served (FCFS) Scheduling

Theoretical Background

"**First-come first-served**" scheduling algorithm, as the name suggests, the process that arrives first gets executed first, or we can say that the process that requests the CPU first gets the CPU allocated first.

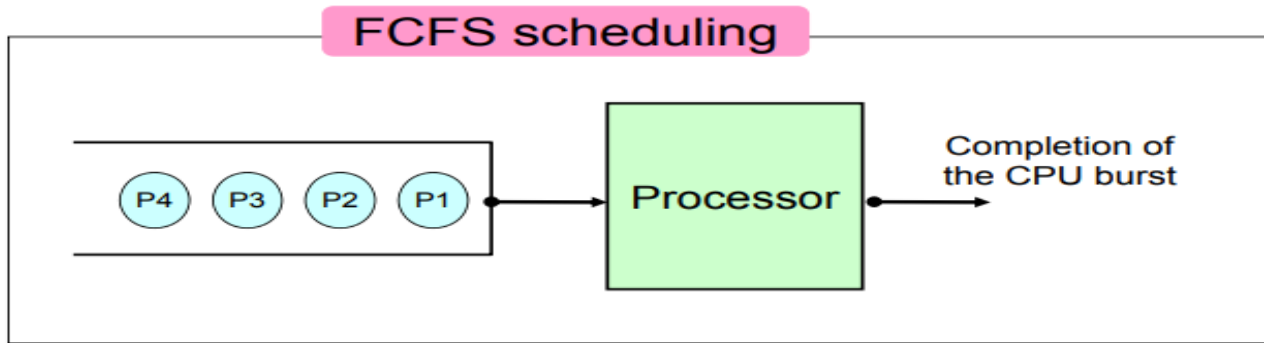
FCFS Scheduling is a **non-preemptive algorithm**, meaning once a process starts running, it **cannot be stopped until** it voluntarily relinquishes the CPU, typically **when it terminates or performs I/O**. This method schedules processes in the order they arrive, without considering priority or other factors(arrival time and burst time).

How Does FCFS Work?

The mechanics of FCFS are straightforward:

1. **Arrival:** Processes enter the system and are placed in a queue in the order they arrive.
2. **Execution:** The CPU takes the first process from the front of the queue, executes it until it is complete, and then removes it from the queue.
3. **Repeat:** The CPU takes the next process in the queue and repeats the execution process.

This continues until there are no more processes left in the queue



Advantage and Disadvantage

Advantages of FCFS

- **The simplest and basic** form of CPU Scheduling algorithm
- Every **process gets a chance to execute in the order of its arrival**. This ensures that no process is arbitrarily prioritized over another.
- **Easy to implement**, it doesn't require complex data structures(just order the process according to arrival time).
- Since processes **are executed in the order they arrive**(fair in order), there's no risk of starvation
- It is well-suited for **batch systems** where the **longer time periods for each process are often acceptable**.

Disadvantages of FCFS

- It is a **Non-Preemptive CPU scheduling algorithm**, so after the process has been allocated to the CPU, it will never release the CPU until it finishes executing.
- The **Average Waiting Time is high**.
- **Short processes** that are at the back of the queue **have to wait for the long process at the front to finish**.
- Not an ideal technique **for time-sharing systems**.
- **Has Convoy Effect**(that Occure when very large process enter at the beginning the other process will wait long time)
- Because of its simplicity, **FCFS is not very efficient**.

Real Applications and Use case

- **Retail:** When stores offer limited-edition products or flash sales, items are usually sold to customers in the order they arrive.
- **Event Ticketing:** Tickets for concerts, conferences, and other popular events are often sold to the first people who book.
- **Customer Service:** Many businesses handle customer requests in the order they are received.
- **Gaming:** Beta versions of new games are often given to the first players who sign up.

3.Implementation Details Section

The algorithm follows the steps:

1. Sort processes by arrival time(this algorithm is based on(arrival time and burst time only).

```
processes.sort(Comparator.comparingInt(p -> p.arrivalTime));
```

2. Initialize the system clock to zero.

```
int time = 0;
int totalburstTime = 0;
int totalWait = 0;
int totalTurn = 0;
int totalResponse = 0;
int idleTime = 0;
int CONTEXT_SWITCH = 1;
```

3. If CPU is idle and no process has arrived → update the time to the first process that exists with the smallest arrival time(add this time to idleTime to calculate CPU Utilization).

```

if (time < p.arrivalTime) {
    idleTime += (p.arrivalTime - time);
    time = p.arrivalTime;
}

```

4. Check if it is the first process or not add Context Switch To idleTime and CPUTime.

```

if (i > 0) {
    time += CONTEXT_SWITCH;
    idleTime += CONTEXT_SWITCH;
}

```

5. Assign the start time.

```

p.startTime = time;

```

6. Execute the process and update the time.

```

p.completionTime = time + p.burstTime;
p.turnaroundTime = p.completionTime - p.arrivalTime;
p.waitingTime = p.startTime - p.arrivalTime;
p.responseTime = p.waitingTime;

time = p.completionTime;
totalburstTime += p.burstTime;
totalWait += p.waitingTime;
totalTurn += p.turnaroundTime;
totalResponse += p.responseTime;

```

7. Calculate metrics.

```
System.out.printf(
    format: "%-5s %-12d %-10d %-10d %-14d %-14d %-14d %-14d\n",
    p.pid, p.arrivalTime, p.burstTime, p.startTime,
    p.completionTime, p.waitingTime,
    p.turnaroundTime, p.responseTime
);
```

8. Compute averages and CPU utilization.

```
double avgWait = (double) totalWait / processes.size();
double avgTurn = (double) totalTurn / processes.size();
double avgResponse = (double) totalResponse / processes.size();
double cpuUtil = (double) totalburstTime / (totalburstTime + idleTime);
double throughput = (double) processes.size() / time;

double fairness = 0;
for (Process p : processes)
    fairness += Math.pow(p.waitingTime - avgWait, 2);

fairness /= processes.size();
```

9. Display results.

```
System.out.println();
System.out.println("Average Waiting Time = " + avgWait);
System.out.println("Average Turnaround Time = " + avgTurn);
System.out.println("Average Response Time = " + avgResponse);
System.out.println("CPU Utilization = " + cpuUtil + "%");
System.out.println("Throughput = " + throughput + " processes/unit");
System.out.println("Fairness (Waiting Time Variance) = " + fairness);
```

```
System.out.printf(
    format: "%-5s %-12s %-10s %-10s %-14s %-14s %-14s %-14s\n",
    ...args: "PID", "arrivalTime", "burstTime", "startTime",
    "completionTime", "waitingTime", "turnaroundTime", "responseTime"
);
```

4. Test Cases output

Evaluation and Impact on Performance of System

```
-----
First-Come-First-Serve Algorithm Scheduler Output:
-----
```

PID	arrivalTime	burstTime	startTime	completionTime	waitingTime	turnaroundTime	responseTime
P1	0	6	0	6	0	6	0
P2	1	4	7	11	6	10	6
P3	2	7	12	19	10	17	10
P4	3	1	20	21	17	18	17
P5	5	4	22	26	17	21	17
P6	6	7	27	34	21	28	21

Gantt Chart:
| P1(0-6) | P2(7-11) | P3(12-19) | P4(20-21) | P5(22-26) | P6(27-34) |

Average Waiting Time = 11.833
Average Turnaround Time = 16.667
Average Response Time = 11.833
CPU Utilization = 85.294%
Throughput = 0.176 processes/unit
Fairness (Waiting Time Variance) = 52.472

5. Challenges Faces and it's Solution

- long average waiting times solution for it use preemptive algorithms
- Convoy effect solution for it use it for short process
- Idle time calculation

Shortest-Job-First (SJF) Scheduling

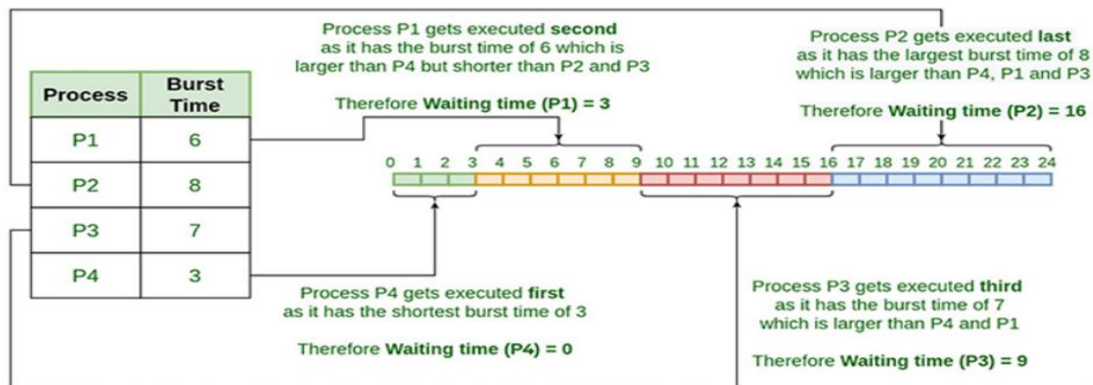
Theoretical Background

The shortest job first (SJF) or shortest job next is a scheduling policy that selects the waiting process **with the smallest execution time to execute next**. SJN, also known as Shortest Job Next (SJN), can be preemptive or non-preemptive.

Characteristics of SJF Scheduling:

- Shortest Job First has the advantage of **having a minimum average waiting time** among all scheduling algorithms.
- It is a **Greedy Algorithm**(choose always the best for now, not for the future).
- **It may cause starvation if shorter processes keep coming**. This problem can be solved using the concept of **ageing**(increasing the importance of the process waiting for a long time).
- **It is practically infeasible as the Operating System may not know burst times and therefore may not sort them**. While it is not possible to predict execution time, several methods can be used to estimate the execution time for a job, such as a weighted average of previous execution times.

Shortest Job First (SJF) Scheduling Algorithm



Shortest Job First Scheduling Algorithm

Advantage and Disadvantage

Advantages of SJF:

- SJF is better than the First-Come-First-Served (FCFS) algorithm as it **reduces the average waiting time**.
 - SJF is generally used **for long-term scheduling**
 - **It is suitable for jobs running in batches, where run times are already known.**
 - SJF is probably optimal **in terms of average turnaround time**.
-

Disadvantages of SJF:

- SJF may **cause very long turnaround times or starvation**.
- In an SJF job, **completion time must be known earlier, but sometimes it is hard to predict**.
- **Sometimes, it is complicated to predict the length of the upcoming CPU request.**

Real Applications and Use case

- **Operating Systems:** In many operating systems, SJF is used to make the best use of the CPU by always running the process that will finish the fastest.
- **Online Delivery:** In delivery apps, the nearest or quickest order is handled first.
- **Batch Processing:** SJF works very well in batch processing systems where many jobs are handled together and users are not waiting for immediate results. Since time is not critical, shorter jobs are completed first to improve overall performance.
- **Dynamic Systems:** Although SJF gives the lowest average waiting time, it can cause problems such as starvation for long jobs. It also depends on knowing how long tasks will take, which is not always possible. For this reason, SJF is not ideal for real-time systems.

3.Implementation Details Section

1. Sort All process with arrival time.

```
processes.sort(Comparator.comparingInt(p -> p.arrivalTime));
```

2. Initialize the system clock to zero and array Completion to check All process is executed.

```

int n = processes.size();
int completed = 0;
int time = 0;
int idleTime = 0;
int CONTEXT_SWITCH = 1;
boolean[] isCompleted = new boolean[n];
Process prev = null;

```

3. Find the process with Smallest burst and arrival Time(make variable minburstTime to compare and find the smallest num and inialized with Integer.MAX_VALUE and compare all process and find the smallest if two process has same burst time choose the smallest arrival time).

```

int idx = -1;
int minburstTime = Integer.MAX_VALUE;

for (int i = 0; i < n; i++) {
    Process p = processes.get(i);
    if (p.arrivalTime <= time && !isCompleted[i]) {
        if (p.burstTime < minburstTime) {
            minburstTime = p.burstTime;
            idx = i;
        } else if (p.burstTime == minburstTime) {
            if (p.arrivalTime < processes.get(idx).arrivalTime) {
                idx = i;
            }
        }
    }
}

```

4. When the process check if the time of process is come or not if it not come update CPUTime and idleTime to start

```

if (idx != -1) {
    Process p = processes.get(idx);

    if (time < p.arrivalTime) {
        idleTime += p.arrivalTime - time;
        time = p.arrivalTime;
    }
}

```

5. Check if it is not the first process to update idleTime and CPUTime with ContextSwitch Time.

```

if (prev != null && prev.pid != p.pid) {
    time += CONTEXT_SWITCH;
    idleTime += CONTEXT_SWITCH;
}

```

6. Execute the process and update times

```

p.startTime = time;
p.completionTime = time + p.burstTime;
p.turnaroundTime = p.completionTime - p.arrivalTime;
p.waitingTime = p.startTime - p.arrivalTime;
p.responseTime = p.waitingTime;
time = p.completionTime;
isCompleted[idx] = true;
completed++;

```

7. Calculate metrics.


```

System.out.printf(
    format: "%-5s %-12d %-10d %-10d %-14d %-14d %-14d %-14d\n",
    p.pid, p.arrivalTime, p.burstTime, p.startTime,
    p.completionTime, p.waitingTime,
    p.turnaroundTime, p.responseTime
);

```

8. Compute averages and CPU utilization.

```

int totalburstTime = 0, totalWait = 0, totalTurn = 0, totalResponse = 0;
double fairness = 0;

for (Process p : processes) {
    totalburstTime += p.burstTime;
    totalWait += p.waitingTime;
    totalTurn += p.turnaroundTime;
    totalResponse += p.responseTime;
}

double avgWait = (double) totalWait / n;
double avgTurn = (double) totalTurn / n;
double avgResponse = (double) totalResponse / n;
double cpuUtil = (double) totalburstTime / (totalburstTime + idleTime) * 100;
double throughput = (double) n / time;

for (Process p : processes)
    fairness += Math.pow(p.waitingTime - avgWait, b: 2);
    fairness /= processes.size(); // Variance

```

9. Display results.

```
System.out.println();
GanttEntry.printGanttChart(ganttEntries);
System.out.println();
System.out.println("Average Waiting Time      = " + avgWait);
System.out.println("Average Turnaround Time    = " + avgTurn);
System.out.println("Average Response Time      = " + avgResponse);
System.out.println("CPU Utilization            = " + cpuUtil + "%");
System.out.println("Throughput (proc/unit time) = " + throughput);
System.out.println("Fairness (Waiting Variance) = " + fairness);
```

4. Test Cases output

Shortest-Job-First Algorithm Scheduler Output:

PID	arrivalTime	burstTime	startTime	completionTime	waitingTime	turnaroundTime	responseTime
P1	0	6	0	6	0	6	0
P4	3	1	7	8	4	5	4
P2	1	4	9	13	8	12	8
P5	5	4	14	18	9	13	9
P3	2	7	19	26	17	24	17
P6	6	7	27	34	21	28	21

Gantt Chart:

| P1(0-6) | P4(7-8) | P2(9-13) | P5(14-18) | P3(19-26) | P6(27-34) |

Average Waiting Time = 9.833
Average Turnaround Time = 14.667
Average Response Time = 9.833
CPU Utilization = 85.294%
Throughput (proc/unit time) = 0.176
Fairness (Waiting Variance) = 51.806

5.Challenges Faces and it's Solution

- Idle time calculation
- **complicated to predict the length of the upcoming CPU request**

Solution for predict the length

- Could ask the user
- Estimate →the next CPU burst will be similar in length to the previous ones.

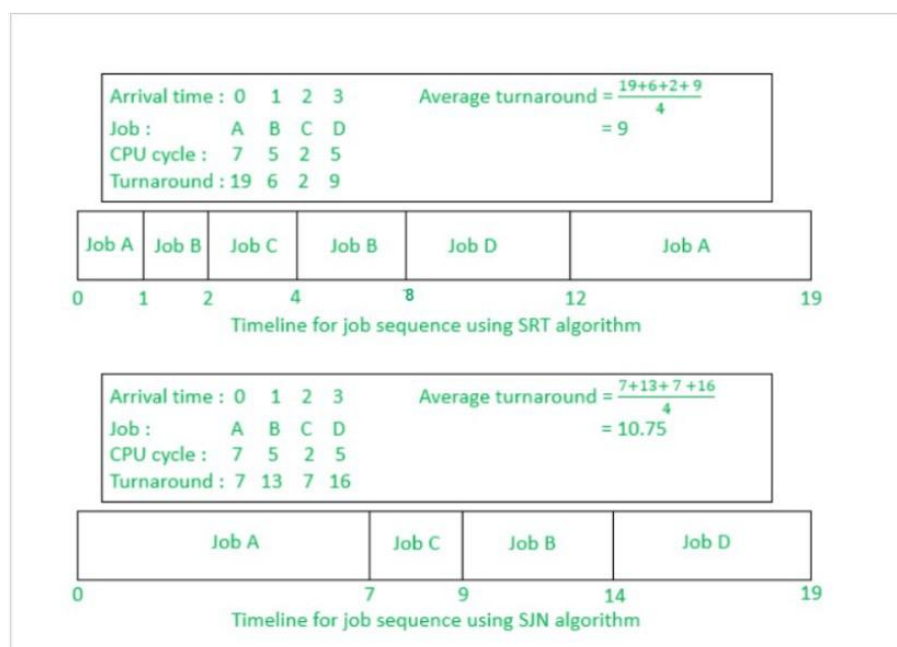
Shortest Remaining Time First Scheduling (SRT)

2.Algorithms Used

Mechanism of Algorithm

Shortest Remaining Time First (SRT) is the preemptive version of Shortest Job Next (SJN) algorithm, where the processor is allocated to the job closest to completion.

This algorithm requires advanced concept and knowledge of CPU time required to process the job in an interactive system, And hence can't be implemented there. But, in a batch system where it is desirable to give preference to short jobs, SRT algorithm is used.



Advantage and Disadvantage

Advantages of SRT Scheduling

- SRT reduces the average waiting time by giving priority to processes with the shortest remaining execution time.
- Efficient for Fast Procedures: Shorter processes are completed faster, improving the system's overall responsiveness.
- Ideal for Time-Critical Systems: It ensures that time-sensitive processes are carried out promptly.

The cons of SRT

- Starvation of long processes: processes with longer time may keep being delayed if shorter process keeps coming.
- Difficult to Predict Burst Times: It is challenging to accurately predict process burst times, which affects scheduling decisions.
- High Overhead: Frequent context switching can slow down a system's performance.
- Unsuitable for Real-Time Systems: tasks may be delayed by frequent preemptions.

Real Applications and Use case

The Completely Fair Scheduler (CFS) in the Linux kernel, introduced in 2007, approximates aspects of shortest remaining time (SRT) scheduling to enhance desktop responsiveness by favoring interactive tasks with short CPU bursts through lower virtual runtime values for processes that frequently sleep or wait on I/O. This mechanism ensures quicker response times for user-facing applications without explicit burst time predictions, as CFS selects the task with the smallest virtual runtime for execution.

In Windows NT-based systems, the scheduler employs dynamic priority boosts for interactive processes, effectively prioritizing those with short CPU bursts to maintain system responsiveness. Threads waiting on user input or I/O receive temporary priority elevations, mimicking SRT behavior by allowing short, foreground tasks to preempt longer-running ones until their bursts complete.

Variants of SRT have been adapted for soft real-time systems in hypervisors like Xen through the SRT-Xen scheduler, which prioritizes virtual machines with shorter remaining execution times when burst estimates are available, improving latency for multimedia workloads without compromising fairness for non-real-time tasks. This approach supports soft real-time priorities in virtualized environments, such as data centers handling bursty, response-sensitive applications.

3.Implementation Details Section

Overall Logic

Step 1: processes with arrival time and burst time as inputs.

Step 2: remaining time, current time = 0, and counters are Initialized.

Step 3: At each second or time unit add processes that have arrived into the ready queue.

Step 4: Select the process with the shortest remaining time (preempt if a shorter one arrives).

Step 5: Execute the selected process for 1 unit, reduce its remaining time, and increment current time.

Step 6: If a process completes:

- $\text{Turnaround Time} = \text{Completion Time} - \text{Arrival Time}$
- $\text{Waiting Time} = \text{Turnaround Time} - \text{Burst Time}$

Step 7: Repeat Steps 3–6 until all processes complete.

Step 8: Calculate the average waiting time and turnaround time.

Step 9: Display completion, waiting, and turnaround times for each process, along with averages.

Snippets of Implementation parts in code

finding the Shortest Job (The Core Logic)

```

for(int i =0 ; i<n ; i++){
    Process p = processes.get(i);
    if (p.arrival <= current_time && p.remaining > 0) { //
    if(p.remaining<mintime){
        mintime= p.remaining;
        indx=i;
        found=true;

    }
    else if(p.remaining==mintime){// eariler goes
        if(p.arrival<processes.get(indx).arrival) // if t
            indx=i;
    }
}
}}

```

Handling Completion

```

// ...
if(currentp.remaining==0) {
    completed++;
    currentp.finish=current_time;
    currentp.turnaround=currentp.finish-currentp.arrival;
    currentp.waiting=currentp.turnaround- currentp.burst;
}

```

Excution of process

```

// 3. EXECUTE THE PROCESS
if (currentp.startTime == -1) {
    currentp.startTime = current_time;
    currentp.responseTime = currentp.startTime - currentp.arrivalTime;
}

currentp.remainingTime--;
current_time++;
totalCpuTime++;

```

4.Test Cases output

Evaluation and Impact on Performance of System

```
run:
PID      Arrival Burst   Finish Turnaround      Waiting Response
-----
P1        0         6      20        20           14         0
P2        1         4       9         8           4         1
P3        2         7      28        26          19        19
P4        3         1       5         2           1         1
P5        5         4      14         9           5         5
P6        6         7      36        30          23        23
-----

Average Turnaround Time: 15.83
Average Waiting Time: 11.00
Average Response Time: 8.17
CPU Utilization: 80.56%
Throughput: 0.17 processes/unit time
Total Context Switches: 7
Total Context Switch Time: 7
```

with an Average Waiting Time of 11.00, along with long individual waiting times, especially for large jobs like P3 and P6, is the numerical proof that the SRTF scheduler operates based on its core principles and achieves those very advantages of doing what it is designed to do, namely, reducing the average wait across the entire process set by aggressively prioritizing those processes with the shortest remaining execution time. That immediate high latency for P3 and P6 is the necessary price paid for ensuring the system works efficiently for fast procedures, guaranteeing that shorter, time-sensitive processes are completed faster. This confirms the appropriateness of this design for time-critical systems.

Compared to SJF (when context switch time is zero)

SRT

```
-----
PID      Arrival Burst   Finish Turnaround      Waiting Response
-----
P1        0         6      15        15           9         0
P2        1         4       6         5           1         0
P3        2         7      22        20          13        13
P4        3         1       4         1           0         0
P5        5         4      10         5           1         1
P6        6         7      29        23          16        16
-----

Average Turnaround Time: 11.50
Average Waiting Time: 6.67
Average Response Time: 5.00
CPU Utilization: 100.00%
Throughput: 0.21 processes/unit time
Total Context Switches: 7
Total Context Switch Time: 0
```

SJF

Shortest-Job-First Scheduler Output:

```
-----
PID  arrivalTime  burstTime  startTime  completionTime  waitingTime  turnaroundTime  responseTime
P1   0           6          0           6              0            6              0
P4   3           1          6           7              3            4              3
P2   1           4          7          11             6           10             6
P5   5           4         11          15             6           10             6
P3   2           7         15          22            13           20             13
P6   6           7         22          29            16           23             16
```

Gantt Chart:

| P1(0-6) | P4(6-7) | P2(7-11) | P5(11-15) | P3(15-22) | P6(22-29) |

Average Waiting Time = 7.333
Average Turnaround Time = 12.167
Average Response Time = 7.333
CPU Utilization = 100.000%
Throughput (proc/unit time) = 0.207
Fairness (Waiting Variance) = 30.556

This proves this comparison

Shortest Job First:	Shortest Remaining Job First:
It is a non-preemptive algorithm.	It is a preemptive algorithm.
It involves less overheads than SRJF.	It involves more overheads than SJF.
It is slower in execution than SRJF.	It is faster in execution than SJF.
It leads to comparatively lower throughput.	It leads to increased throughput as execution time is less.
It minimizes the average waiting time for each process.	It may or may not minimize the average waiting time for each process.
It may suffer from priority inversion.	It may suffer from convoy effect.
It involves lesser number of context switching.	It involves higher number of context switching.
Short processes are executed first and then followed by longer processes.	Shorter processes run fast and longer processes show poor response time.

5.Challenges Faces and it's Solution

First thing was handling idle cpu time as if no process had arrived yet or there is a gap between processes the loop would crash or behave unpredictably. So implemented if (!found) to detect when the ready queue is empty in this case just increment the current_time to simulate cpu being idle.

Second challenge faced was zombie process bug which was about selecting process that were already finished because their remaining is 0 was smaller than unfinished processes so added strict condition $p.\text{remaining} > 0$ in selection loop to ensure we only consider active ones

Third challenge and finally was determining when to switch process so time step (second by second) was utilized allowing us to re-evaluate SRT at every second guaranteeing that if a new shorter job arrived it would be picked.

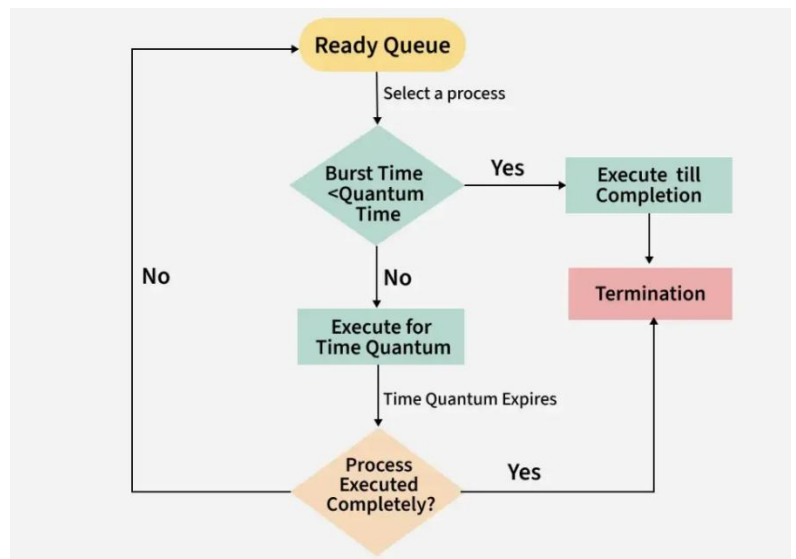
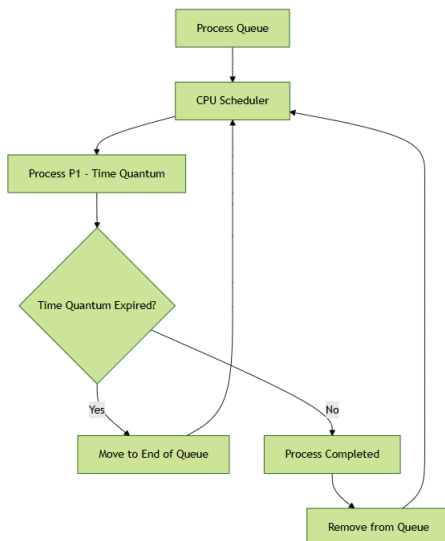
Round Robin (RR)

2.Algorithms Used

Theoretical Background

Round Robin (RR) scheduling is a **preemptive CPU scheduling algorithm** that assigns equal time slices, called time quantum or time slice, to each process in a circular queue. It's one of the most widely used scheduling algorithms in modern operating systems due to its fairness and simplicity.

The algorithm operates on a simple principle: each process gets an equal opportunity to execute for a fixed time period. When a process's time quantum expires, it's moved to the back of the ready queue, and the next process gets CPU access.



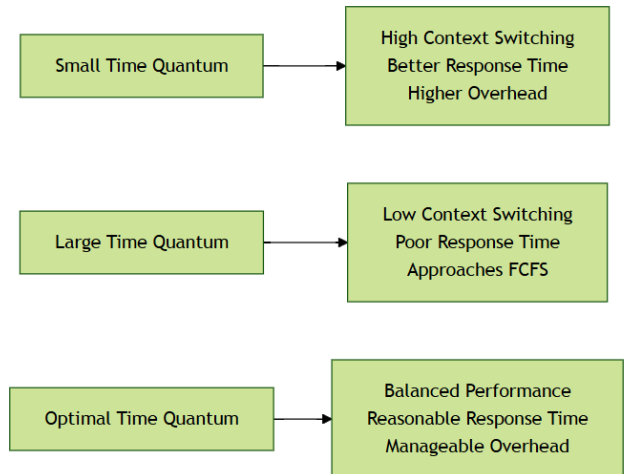
Key Components of Round Robin Scheduling

Time Quantum (Time Slice)

The time quantum is the most critical parameter in Round Robin scheduling. It determines how long each process can execute before being preempted. The choice of time quantum significantly impacts system performance

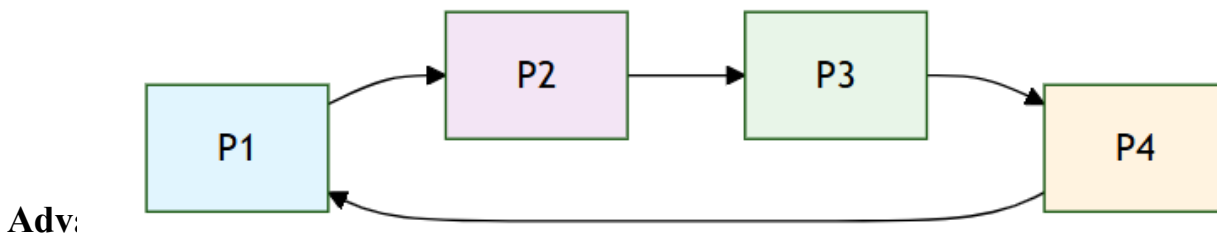
- **Small Time Quantum:** Better response time but higher context switching overhead
- **Large Time Quantum:** Reduced overhead but may lead to poor response time

- **Optimal Time Quantum:** Balances response



Ready Queue Structure

Round Robin uses a **circular queue (First-In-First-Out)** to maintain processes. New processes are added to the tail, and the scheduler picks processes from the head.



Advantages of Round Robin Scheduling

- **Fairness:** Every process gets equal CPU time allocation
- **No Starvation:** All processes eventually get CPU access
- **Good Response Time:** Interactive processes receive quick response
- **Simple Implementation:** Easy to understand and implement
- **Preemptive:** Prevents long processes from monopolizing CPU
- **Time Sharing:** Ideal for multi-user and interactive systems

Disadvantages of Round Robin Scheduling

- **Context Switching Overhead:** Frequent switches consume system resources
- **Poor for Batch Jobs:** Not optimal for CPU-intensive processes
- **Time Quantum Dependency:** Performance heavily depends on quantum selection

- **Average Waiting Time:** Often higher than other algorithms
- **No Priority Consideration:** Treats all processes equally regardless of importance

Real Applications and Use case

Operating Systems Using Round Robin

- **Linux:** Completely Fair Scheduler (CFS) incorporates RR concepts
- **Windows:** Uses round robin for threads at the same priority level
- **Unix:** Traditional Unix systems use RR for time-sharing
- **Embedded Systems:** Real-time systems often use modified RR

3.Implementation Details Section

Data Structures Used

- **ArrayLists as List<Process>** for storing all input processes and it's attributes
- **ArrayLists List<GanttEntry>** for implementing the Gantt chart entries and execution timeline
- **LinkedList as Queue<Process>** to store the processes in ready queue for RR algorithm
- **Array as Boolean[] inQueue** for checking if the processes is already in ready queue or not

Overall Logic

1- Initialization

```
int n = processes.size();
int time = 0;

processes.sort(Comparator.comparingInt(p -> p.arrivalTime));

Queue<Process> readyQueue = new LinkedList<>();
boolean[] inQueue = new boolean[n];

int completed = 0;
int totalContextSwitches = 0;
Process lastProcess = null;

List<GanttEntry> ganttChart = new ArrayList<>();
```

In initialization section :

- Sorting processes by it's arrival time
- Create ready queue and initial other Data Structures

Initial important variables

- completed for counting completed processes
- totalContextSwitch for counting number of switching between processes
- lastProcesses to get the last processes has finished

2- Main Scheduling Loop

While all processes are not completed then

- Add new one which has arrived to ready queue
- If the CPU is in IDLE case, jump forward for the next arrival process and record IDLE case in GanttChart
- If the case is switching between processes then record CS case for Gantt chart

3- Processes Execution

For Each process (Currently Process)

- If it's first time running , then calculate it's response time

```
current.responseTime = current.startTime - current.arrivalTime;
```

- Execution will be for either quantum or the remaining time if it is less than q

```
int execTime = Math.min(quantum, current.remainingTime);  
int executionEnd = time + execTime;
```

- During execution the process checking for new arrival is done and add them to the queue
- Finally record the execution in the Gantt chart

4- Complete and Finish the Queue

- If The process still has remaining time , put it again in the queue
- If it is finished , calculate the Completion time, turnaround time and the waiting time then increment the complete variable

5- Final output

- Printing the Gantt Chart
- Print overall metrics(Avg Waiting time, turnaround time, CPU utilization, cotext switch)
- Print the process table with detailed results

Snippets of Implementation parts in code

Checking CPU IDE

```
// CPU IDLE
if (current == null) {
    int nextArrivalTime = Integer.MAX_VALUE;
    for (Process p : processes) {
        if (p.remainingTime > 0 && p.arrivalTime > time) {
            nextArrivalTime = Math.min(nextArrivalTime, p.arrivalTime);
        }
    }
    if (nextArrivalTime != Integer.MAX_VALUE) {
        ganttChart.add(new GanttEntry(pid: "IDLE", time, nextArrivalTime));
        time = nextArrivalTime;
    } else break;
}

lastProcess = null;
continue;
}
```

Checking Context Switch

```
// Context Switch
if (lastProcess != null && lastProcess != current) {
    ganttChart.add(new GanttEntry(pid: "CS", time, time + contextSwitchTime));
    time += contextSwitchTime;
    totalContextSwitches++;

    // Check for new arrivals during context switch
    for (int i = 0; i < n; i++) {
        Process p = processes.get(i);
        if (p.arrivalTime <= time && !inQueue[i] && p.remainingTime > 0) {
            readyQueue.add(p);
            inQueue[i] = true;
        }
    }
}
}
```

First Response Calculations

```
// First Response time
if (current.startTime == -1) {
    current.startTime = time;
    current.responseTime = current.startTime - current.arrivalTime;
}
```

Execution Calculations and adding to queue if it is not finished

```

gantChart.add(new GanttEntry(current.pid, time - execTime, time));
current.remainingTime -= execTime;

// Add current process back to queue if not finished
if (current.remainingTime > 0) {
    readyQueue.add(current);
} else {
    current.completionTime = time;
    current.turnaroundTime = current.completionTime - current.arrivalTime;
    current.waitingTime = current.turnaroundTime - current.burstTime;
    completed++;
}

```

4. Test Cases output Q=4

Gantt Chart:
 | P1(0-4) | CS(4-5) | P2(5-9) | CS(9-10) | P3(10-14) | CS(14-15) | P4(15-16) | CS(16-17) | P1(17-19) | CS(19-20) | P5(20-24) | CS(24-25) | P6(25-29) | CS(29-30) | P3(30-33) | CS(33-34) | P6(34-37) |

Round Robin 's Metrix Output:

Average Turnaround Time: 20.17
 Average Waiting Time: 15.33
 Average Response Time: 9.67
 CPU Utilization: 78.38%
 Throughput: 0.16 processes/unit time
 Total Context Switches: 8
 Total Context Switch Time: 8

Process Table Output:

PID	Arrival Time	Burst Time	Start Time	Complete Time	Turnaround Time	Waiting Time	Response Time	Priority
P1	0	6	0	19	19	13	0	1
P2	1	4	5	9	8	4	4	2
P3	2	7	10	33	31	24	8	3
P4	3	1	15	16	13	12	12	4
P5	5	4	20	24	19	15	15	4
P6	6	7	25	37	31	24	19	3

Evaluation and Impact on Performance of System

- Round Robin provided fairness response time
- However , Performance was impact by high waiting times, large turnaround times and reduced CPU utilization due to context switch overhead
- The choice of q is critical : too small increase overhead, too large reduce fairness

Test Cases output Q=3 Smaller

Round Robin Algorithm Scheduler Output:

Gantt Chart:
 | P1(0-3) | CS(3-4) | P2(4-7) | CS(7-8) | P3(8-11) | CS(11-12) | P4(12-13) | CS(13-14) | P1(14-17) | CS(17-18) | P5(18-21) | CS(21-22) | P6(22-25) | CS(25-26) | P2(26-27) | CS(27-28) | P3(28-31) | CS(31-32) | P5(32-33) | CS(33-34) | P6(34-37) | CS(37-38) | P3(38-39) | CS(39-40) | P6(40-41) |

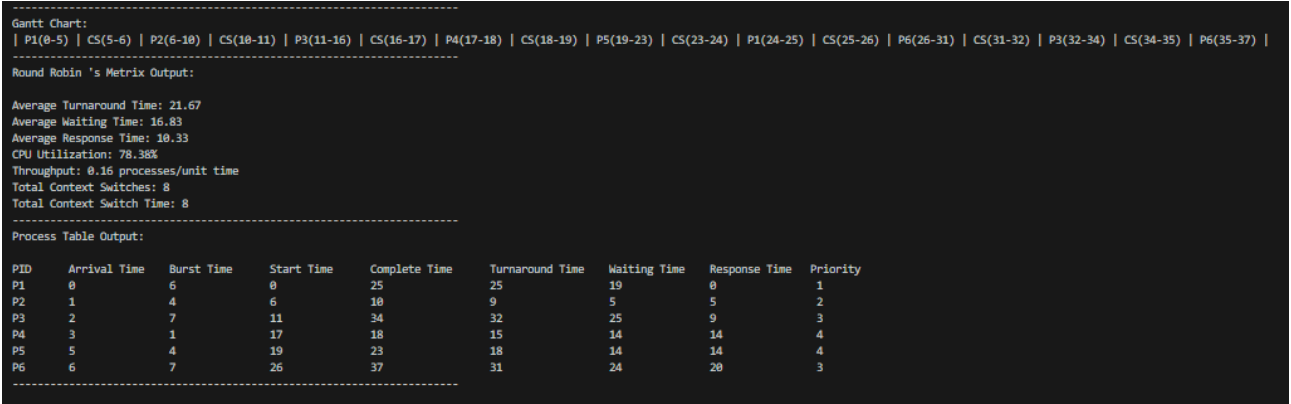
Round Robin 's Metrix Output:

Average Turnaround Time: 25.50
 Average Waiting Time: 20.67
 Average Response Time: 7.83
 CPU Utilization: 70.73%
 Throughput: 0.15 processes/unit time
 Total Context Switches: 12
 Total Context Switch Time: 12

Process Table Output:

PID	Arrival Time	Burst Time	Start Time	Complete Time	Turnaround Time	Waiting Time	Response Time	Priority
P1	0	6	0	17	17	11	0	1
P2	1	4	4	27	26	22	3	2
P3	2	7	8	39	37	30	6	3
P4	3	1	12	13	10	9	9	4
P5	5	4	18	33	28	24	13	4
P6	6	7	22	41	35	28	16	3

Test Cases output Q=5 larger



Challenges Faces and it's Solution

- Time Quantum Selection
Time Quantum Dependency: Performance heavily depends on quantum selection

Time Quantum Selection Guidelines

Choosing optimal time quantum depends on several factors:

System Type	Recommended Quantum	Reasoning
Interactive Systems	10-100ms	Good response time for user interaction
Server Systems	100-500ms	Balance between throughput and responsiveness
Batch Systems	500ms-2s	Minimize context switching overhead
Real-time Systems	1-10ms	Meet strict timing requirements

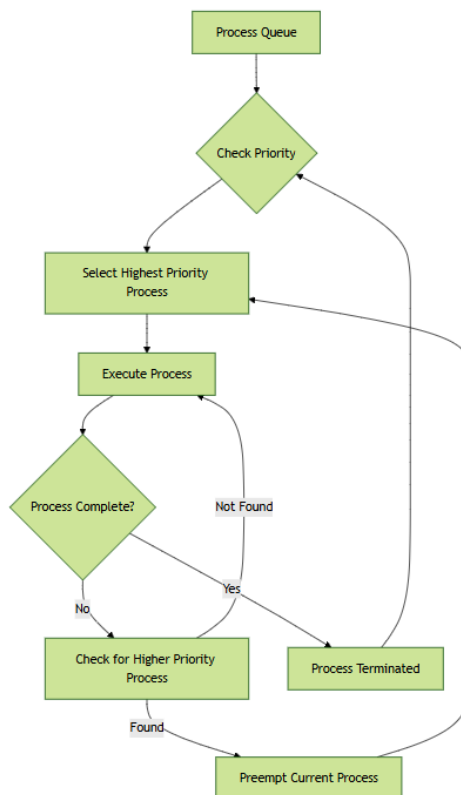
Priority Scheduling

2.Algorithms Used

Theoretical Background

Priority scheduling is a CPU scheduling algorithm where each process is assigned a priority value, and the CPU is allocated to the process with the highest priority. This algorithm ensures that critical system processes and time-sensitive applications receive CPU resources before less important tasks.

The priority scheduling algorithm operates on a simple principle: higher priority processes execute first. When multiple processes are ready for execution, the scheduler selects the one with the highest priority value (lowest numerical value)



Types of Priority Scheduling

Non-Preemptive Priority Scheduling

In non-preemptive priority scheduling, once a process starts executing, it continues until completion or until it voluntarily gives up the CPU. Even if a higher priority process arrives, the currently running process is not interrupted.

Characteristics:

- Simple implementation and lower overhead
- No context switching during process execution
- May cause starvation of low-priority processes
- Better for batch processing systems

Preemptive Priority Scheduling

In **preemptive priority scheduling**, if a new process with higher priority arrives while another process is executing, the currently running process is preempted and moved back to the ready queue.

Characteristics:

- More responsive to high-priority processes
- Higher overhead due to frequent context switching
- Better for interactive and real-time systems
- Requires careful priority assignment to prevent starvation

Priority Assignment Methods

Static Priority Assignment

Priorities are assigned when processes are created and remain constant throughout execution. Common methods include:

- **User-defined priorities:** Users specify priority levels
- **Process type-based:** System processes get higher priority than user processes

- **Memory requirements:** Processes with smaller memory footprint get higher priority

Dynamic Priority Assignment

Priorities change during process execution based on various factors:

- **Aging technique:** Gradually increase priority of waiting processes
- **Resource usage:** Adjust priority based on CPU usage patterns
- **Waiting time:** Increase priority for processes waiting long

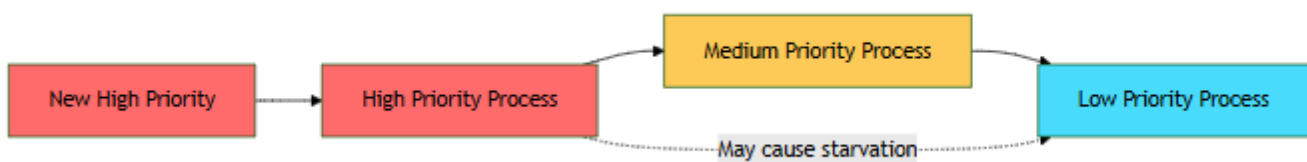
Advantage and Disadvantage

Advantages of Priority Scheduling

- **Flexible scheduling:** Allows prioritization of critical processes
- **Good response time:** High-priority processes get quick response
- **Suitable for real-time systems:** Critical tasks can be prioritized
- **Easy to understand:** Simple concept and implementation

Disadvantages of Priority Scheduling

- **Starvation problem:** Low-priority processes may never execute
- **Priority inversion:** High-priority process waiting for low-priority resource
- **Complexity in priority assignment:** Difficult to determine optimal priorities
- **Context switching overhead:** Frequent preemption increases overhead



Real Applications and Use case

Operating Systems Using Priority Scheduling

- **Windows:** Uses priority-based scheduling with 32 priority levels

- **Linux:** Completely Fair Scheduler (CFS) with nice values
- **Real-time systems:** Rate Monotonic Scheduling (RMS)
- **Embedded systems:** Fixed-priority preemptive scheduling

3.Implementation Details Section

Data Structures Used

- **ArrayLists as List<Process>** for storing all input processes and it's attributes
- **ArrayLists List<GanttEntry>** for implementing the Gantt chart entries and execution timeline
- **LinkedList as Queue<Process>** to store the processes in ready queue for RR algorithm used only when multiple processes share the same priority
- **Maps** for tracking the first start time , last end time , burst time and arrival time
- **Sets** to count completed processes when computing through put

Overall Logic

1- Initialization

- Reset all process (remain time , start time, wait, turnaround)

Initial important variables

- completed for counting completed processes
- totalContextSwitch for counting number of switching between processes
- lastProcesses to get the last processes has finished

2- Main Loop

- At Each time unit collect all arrived and unfinished processes and if non available one , CPU Get IDLE Case

3- Priority Selection

- For available processes we pick the highest one & if multiple has the same priority use RR

4- Preemptive Algorithm

- If higher priority arrive current one is pre-empted immediately & CS case is added to Gantt chart

5- RR inside Same priority Level

- Same priority processes are added to a queue ordered by arrival
- Each one gets a q time
- If it is unfinished enter it again to the queue
- Still preemptive if a higher one appear

6- Calculate Performance Metrics

- When a process first start , calculate the response time
- When it is finished compute the completion time, turnaround time , waiting time

7- After Loop

- Final metrics using Gantt Chart
- CPU Utilization
- Throughput
- AVG waiting time
- AVgTurnaround time
- AVG Response time
- Total Context Switch time

Snippets of Implementation parts in code

Determine Type of Algorithm based on Boolean variable

```
public static void priorityScheduling(List<Process> processes, boolean preemptive) {  
    if (preemptive) {  
        System.out.println(x: "\n=====");  
        System.out.println(x: "Priority (Preemptive) Scheduling:");  
        System.out.println(x: "=====");  
        priorityPreemptiveWithRoundRobin(processes);  
    } else {  
        System.out.println(x: "\n=====");  
        System.out.println(x: "Priority (Non-Preemptive) Scheduling:");  
        System.out.println(x: "=====");  
        priorityNonPreemptive(processes);  
    }  
}
```

find highest priority available (lower number = higher priority)

```
available.sort(Comparator.comparingInt(p -> p.currentPriority));  
int highest = available.get(index: 0).currentPriority;
```

Preemptive Single Process Execution

```
if (samePriority.size() == 1) {  
    Process cur = samePriority.get(index: 0);  
  
    if (lastProcess != null && lastProcess != cur) {  
        gantt.add(new GanttEntry(pid: "CS", time, time + contextSwitchTime)  
        time += contextSwitchTime;  
        totalContextSwitches++;  
    }  
  
    if (cur.startTime == -1) {  
        cur.startTime = time;  
        cur.responseTime = cur.startTime - cur.arrivalTime;  
    }  
  
    gantt.add(new GanttEntry(cur.pid, time, time + 1));  
    cur.remainingTime -= 1;  
    time += 1;  
  
    for (Process p : processes) {  
        if (p != cur && p.arrivalTime <= time && p.remainingTime > 0) {  
            p.waitingTimeCounter++;  
        }  
    }  
  
    if (cur.remainingTime == 0) {  
        cur.completionTime = time;  
        cur.turnaroundTime = cur.completionTime - cur.arrivalTime;  
        cur.waitingTime = cur.turnaroundTime - cur.burstTime;  
        completed++;  
    }  
  
    lastProcess = cur;  
}
```

When only one process exists at the highest priority level, the scheduler runs it using preemptive execution :

- 1- Context switch is added if the CPU switches between processes
- 2- If the process is starting for first time , it's response time is calculated
- 3- The process executes for one time unit only allowing preemptive if a higher- priority process arrives later
- 4- All other ready processes increase their waiting counter (for aging solution)
- 5- If the process finishes then the completion and turnaround waiting time are calculated
- 6- The executed process becomes the lastProcess for contextSwitch track

Round Robin for Same Priority Process

- RR queue is created and ordered by arrival time , then by PID to keep execution fair
- Each process gets one q of CPU time
- Before running each process the scheduler checks if higher priority one has arrived and if yes it stops RR and return to main loop
- Context switch is added if the CPU switches between processes
- If the process is starting for first time , it's response time is calculated
- After q , new process of same priority are added to the queue
- If the process still has remaining time, it added again to the queue, other wise it's performance metrics are updated
- During RR if higher priority process appears, the scheduler stops RR and handles it

```
List<Process> check = new ArrayList<>();
for (Process p : processes)
    if (p.arrivalTime <= time && p.remainingTime > 0)
        check.add(p);
check.sort(Comparator.comparingInt(p -> p.currentPriority));
if (!check.isEmpty() && check.get(index: 0).currentPriority < highest) {
    break;
}

if (lastProcess != null && lastProcess != cur) {
    gantt.add(new GanttEntry(pid: "CS", time, time + contextSwitchTime));
    time += contextSwitchTime;
    totalContextSwitches++;
}

if (cur.startTime == -1) {
    cur.startTime = time;
    cur.responseTime = cur.startTime - cur.arrivalTime;
}

int t = Math.min(quantum, cur.remainingTime);

gantt.add(new GanttEntry(cur.pid, time, time + t));
cur.remainingTime -= t;
time += t;

for (Process p : processes) {
    if (p.arrivalTime <= time && p.remainingTime > 0 && p.currentPriority == highest && !q.contains(p) && p != cur) {
        q.add(p);
    }
}

if (cur.remainingTime > 0) {
    q.add(cur);
} else {
    cur.completionTime = time;
    cur.turnaroundTime = cur.completionTime - cur.arrivalTime;
    cur.waitingTime = cur.turnaroundTime - cur.burstTime;
    completed++;
}

lastProcess = cur;

check.clear();
for (Process p : processes)
    if (p.arrivalTime <= time && p.remainingTime > 0)
        check.add(p);
check.sort(Comparator.comparingInt(p -> p.currentPriority));
if (!check.isEmpty() && check.get(index: 0).currentPriority < highest) {
    break;
}
```

Non Preemptive Process Execution

- The scheduler selects the ready process with the highest priority
- If multiple processes have the same priority, the one with the earliest arrival time is chosen
- Once a process starts, it runs until completion with no interruption
- Context switch is added if the CPU switches between processes
- If the process is starting for first time, its response time is calculated
- Final metrics performance are calculated

```
while (completed < n) {
    Process current = null;
    int highestPriority = Integer.MAX_VALUE;
    int earliestArrival = Integer.MAX_VALUE;

    for (Process p : processes) {
        if (p.arrivalTime <= time && p.remainingTime > 0) {
            if (p.currentPriority < highestPriority) {
                highestPriority = p.currentPriority;
                current = p;
                earliestArrival = p.arrivalTime;
            } else if (p.currentPriority == highestPriority && p.arrivalTime < earliestArrival) {
                current = p;
                earliestArrival = p.arrivalTime;
            }
        }
    }

    if (current == null) {
        int nextArrivalTime = Integer.MAX_VALUE;
        for (Process p : processes) {
            if (p.remainingTime > 0 && p.arrivalTime > time) {
                nextArrivalTime = Math.min(nextArrivalTime, p.arrivalTime);
            }
        }
        if (nextArrivalTime == Integer.MAX_VALUE) break;
        ganttChart.add(new GanttEntry(pid: "IOLE", time, nextArrivalTime));
        time = nextArrivalTime;
        lastProcess = null;
        continue;
    }

    if (lastProcess != null && lastProcess != current) {
        ganttChart.add(new GanttEntry(pid: "CS", time, time + contextSwitchTime));
        time += contextSwitchTime;
        totalContextSwitches++;
    }

    if (current.startTime == -1) {
        current.startTime = time;
        current.responseTime = current.startTime - current.arrivalTime;
    }

    int execTime = current.remainingTime;
    ganttChart.add(new GanttEntry(current.pid, time, time + execTime));
    current.remainingTime = 0;
    time += execTime;

    current.completionTime = time;
    current.turnaroundTime = current.completionTime - current.arrivalTime;
    current.waitingTime = current.turnaroundTime - current.burstTime;
    completed++;

    lastProcess = current;
}
```


4.Test Cases output

Test Cases output Preemptive

```
Priority Algorithm Scheduler Output:
=====
Priority (Preemptive) Scheduling:
=====
Gantt Chart:
| P1(0-1) | P1(1-2) | P1(2-3) | P1(3-4) | P1(4-5) | P1(5-6) | C5(6-7) | P2(7-8) | P2(8-9) | P2(9-10) | P2(10-11) | C5(11-12) | P3(12-16) | C5(16-17) | P6(17-21) | C5(21-22) | P3(22-25) | C5(25-26) | P6(26-29) | C5(29-30) | P4(30-31) | C5(31-32) | P5(32-36) |
=====
Metrics (calculated from Gantt):
Total time: 36
Context switches: 7
CPU busy time: 29
CPU Utilization: 80.56%
Throughput: 0.1667 process/unit-time
Average Turnaround Time: 20.17
Average Waiting Time: 15.33
Average Response Time: 13.50
=====
Process Table:
PID  Arrival Burst  Start  Completion  Turnaround  Waiting  Response
P1    0      6      0       6          6          0          0
P2    1      4      7      11         10          6          6
P3    2      7     12      25         23         16         10
P4    3      1     30      31          28         27         27
P5    5      4     32      36          31         27         27
P6    6      7     17      29          23         16         11
```

Evaluation and Impact on Performance of System

Ensure Fast response for critical processes but introduce overhead and delays for low-priority tasks , leading to possible starvation

Test Cases output Non-Preemptive

```
=====
Priority (Non-Preemptive) Scheduling:
=====
Gantt Chart:
| P1(0-6) | C5(6-7) | P2(7-11) | C5(11-12) | P3(12-19) | C5(19-20) | P6(20-27) | C5(27-28) | P4(28-29) | C5(29-30) | P5(30-34) |
=====
Metrics (calculated from Gantt):
Total time: 34
Context switches: 5
CPU busy time: 29
CPU Utilization: 85.29%
Throughput: 0.1765 process/unit-time
Average Turnaround Time: 18.17
Average Waiting Time: 13.33
Average Response Time: 13.33
=====
Process Table:
PID  Arrival Burst  Start  Completion  Turnaround  Waiting  Response
P1    0      6      6       6          6          0          0
P2    1      4      7      11         10          6          6
P3    2      7     12      19          17         10         10
P4    3      1     28      29          26         25         25
P5    5      4     30      34          29         25         25
P6    6      7     20      27          21         14         14
```

Evaluation and Impact on Performance of System

Improve throughput and efficiency, but high-priority processes may wait longer if low-priority one is already running

5.Challenges Faces and it's Solution

Starvation Problem and it's Solution Aging Technique:

Starvation problem: Low-priority processes may never execute

Aging Technique

The **aging technique** is the most common solution to prevent starvation in priority scheduling. It gradually increases the priority of processes that have been waiting for a long time

Aging TestCase and it's output

Test Case:

```
//For test Aging solve Priority algorithm
public static List<Process> getPrioritySchedulerStarvationCase() {
    List<Process> processes = new ArrayList<>();
    processes.add(new Process(pid: "P1", arrivalTime: 0, burstTime: 40, priority: 3)); // low priority
    processes.add(new Process(pid: "P2", arrivalTime: 2, burstTime: 4, priority: 2)); // higher priority
    processes.add(new Process(pid: "P3", arrivalTime: 4, burstTime: 10, priority: 1)); // highest priority
}
```

Without Aging

```
Priority (Preemptive) Scheduling:
Gantt Chart:
| P1(0-1) | P1(1-2) | CS(2-3) | P2(3-4) | CS(4-5) | P3(5-6) | P3(6-7) | P3(7-8) | P3(8-9) | P3(9-10) | P3(10-11) | P3(11-12) | P3(12-13) | P3(13-14) | P3(14-15) | CS(15-16) | P2(16-17) | P2(17-18) | P2(18-19) | CS(19-20) | P1(20-21) | P1(21-22) | P1(22-23) | P1(23-24) | P1(24-25) | P1(25-26) | P1(26-27) | P1(27-28) | P1(28-29) | P1(29-30) | P1(30-31) | P1(31-32) | P1(32-33) | P1(33-34) | P1(34-35) | P1(35-36) | P1(36-37) | P1(37-38) | P1(38-39) | P1(39-40) | P1(40-41) | P1(41-42) | P1(42-43) | P1(43-44) | P1(44-45) | P1(45-46) | P1(46-47) | P1(47-48) | P1(48-49) | P1(49-50) | P1(50-51) | P1(51-52) | P1(52-53) | P1(53-54) | P1(54-55) | P1(55-56) | P1(56-57) | P1(57-58) |
```

With Aging

```
=== Aging Test ===
[Aging] Time 19: P1 oldPriority=3, newPriority=2
Gantt Chart:
| P1(0-4) | CS(4-5) | P3(5-9) | P3(9-13) | P3(13-15) | P2(15-19) | P1(19-23) | P1(23-27) | P1(27-31) | P1(31-35) | P1(35-39) | P1(39-43) | P1(43-47) | P1(47-51) | P1(51-55) |
Process Table:
PID  Arrival  Burst  Start  Completion  Turnaround  Waiting  Response  BasePrio  CurPrio
P1    0        40    0      55          55         15         0          3         2
P2    2         4    15     19          17         13         13         2         2
P3    4         10    5      15          11         1          1          1         1
```

priority inversion problems:

process with high priority want a resource which is holding by a low priority one , so it will wait until releasing the resource and this can block important tasks and reduce system responsiveness

Priority Inheritance Solution

The system temporarily raises the priority of low-priority process which holding the resource so it can finish quickly then once the resource is released , its priority returns to normal allowing the high priority one to continue and complete

Priority Inheritance TestCase

```
// Test Case for Priority Inversion:
// Low priority process holds a shared resource.
// High priority process needs the resource So might get blocked.
// Optional: simulate priority inheritance in scheduler.
//For test priority inheritance solve
public static List<ProcessWithResource> getPrioritySchedulerInversionCase() {
    List<ProcessWithResource> processes = new ArrayList<>();
    resource file = new resource();

    // P1 low priority, owns resource
    processes.add(new ProcessWithResource(pid: "P1", arrivalTime: 0, burstTime: 5, priority: 3, file));

    // P2 high priority, needs resource
    processes.add(new ProcessWithResource(pid: "P2", arrivalTime: 1, burstTime: 3, priority: 1, file));

    // P3 medium priority, no resource
    processes.add(new ProcessWithResource(pid: "P3", arrivalTime: 2, burstTime: 2, priority: 2, res: null));

    return processes;
}
```

Priority Inheritance Output

P1's priority is a rised temporarily to complete execution and release the resource so the p1 can hold the resource and complete execution

Gantt Chart:
| P1(0-4) | CS(4-5) | P2(5-8) | P3(8-10) | P1(10-11) |

Process Table:

PID	Arrival	Burst	Start	Completion	Turnaround	Waiting	Response	BasePrio	CurPrio
P1	0	5	0	11	11	6	0	3	3
P2	1	3	5	8	7	4	4	1	1
P3	2	2	8	10	8	6	6	2	2

c:\Users\Aya Mohamed\Documents\college\OS\Project - Copy

Multilevel Queue

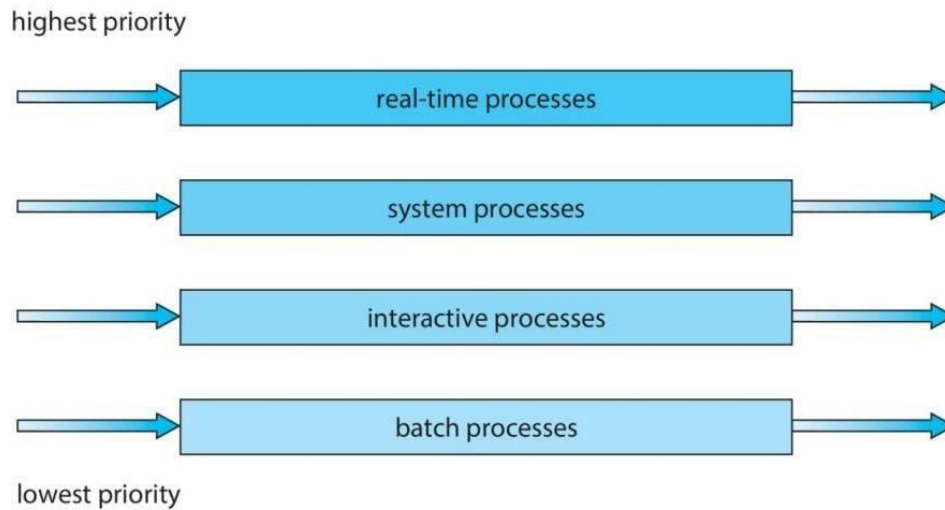
2.Algorithms Used

Theoretical Background

In multilevel queue scheduling, the processes are divided into separate categories, where category has its own distinct characteristics. In most systems, there are three types of processes, the system processes, the foreground processes and the background processes.

The system processes are of the highest priority and need immediate attention. The foreground processes are user processes that are interactive in nature; while the background processes are batch processes. Since, the different categories require different response time and priorities, they require different scheduling algorithms. These processes are assigned to different queues as

shown in the following diagram.



Advantage and Disadvantage

Advantages :

- The main advantage of MLQ is that it allows **setting priorities** between the processes, which is an essential requirement for practical purposes.
- MLQ enables **efficient distribution of the system resources** between the different processes.
- MLQ **reduces the response times** for high priority processes drastically. This makes the overall system more responsive.
- MLQ is highly **customizable** according to the system needs.
- MLQ improves **system performances** and increases throughput.
- It has **low scheduling overheads** in systems with varied process mix

Disadvantages :

- A process in a low priority queue may face **undefined starvation** if there are many high priority queues each having steady streams of processes. This is the main drawback of MLQ.

- Implementation of MLQ is inherently **complex**. It requires proper design and development, failing which the entire scheduling mechanism may break down.
- Assigning fixed priorities to processes makes MLQ an **inflexible** strategy.
- The scheduling is too much dependent on the **fixed priorities**, which are assigned as soon as a process enters the system. Faulty assignment of process priorities hampers performance in big way.

Real Applications and Use case

- **Time-Sharing Operating Systems (e.g., Linux, Windows):** MLQ helps balance user-facing applications (foreground) with system tasks (background), ensuring smooth web browsing and immediate feedback for mouse/keyboard actions while batch jobs run efficiently.
- **Server Environments:** Prioritizes critical services (e.g., database queries, web requests) over less urgent tasks like log processing or scheduled reports, ensuring responsiveness for high-demand services.
- **Interactive Applications:** Users get better response times for text editors, graphical apps, and browsers because these are in high-priority queues.
- **Batch Processing Systems:** Non-interactive tasks like large data computations or system backups are placed in lower-priority queues, running consistently without interrupting interactive users

3.Implementation Details Section

Overall Logic

The principle of execution is as follows –

Let there be n queues numbered from 1 to n having priorities from 1 to n . Each process is permanently assigned to one of these queues. A queue enjoys absolute priority over the queues having lower priorities. Each queue has its own scheduling strategy. At first, queue 1 with priority 1 is checked. If there are processes in it, they execute according to the scheduling strategy of queue 1. If there are no processes in it, then queue 2 is checked. If it has processes, then they are scheduled according to its scheduling algorithm. This continues up to queue n . Thus a process in queue x can gain access to CPU only if queue 1 to queue $(x-1)$ are empty. If a process, say P5, at queue x is executing and a new process, say P16, arrives at a queue having higher priority, P16 immediately pre-empts P5 and starts to execute.

Snippets of Implementation parts in code

Our MLQ code consists of 3 main parts:

1-Process class --> initial process data and constructor is used to create a new Process object and initialize its initial properties.

```
static class Process {
    String id;
    int arrival;
    int burst;
    int remaining;
    int queue; // 0..3

    int startTime = -1;
    int finishTime = -1;
    int responseTime = -1;
    int waitingTime = 0;
    int turnaround = -1;

    public Process(String id, int arrival, int burst, int queue) {
        this.id = id;
        this.arrival = arrival;
        this.burst = burst;
        this.remaining = burst;
        this.queue = queue;
    }
}
```

2- MLQ class

-assuming that the priority of Queues are as following :

Q0 (highest priority) ----> runs using SRT algorithm

Q1 -----> runs using SJF algorithm

Q2 -----> runs using RR algorithm

Q3 (lowest priority) -----> runs using FCFS algorithm

- We check at every time step if any process arrives, places it in its proper queue, and logs the arrival, while the loop continues until all processes are finished.

```
while (true) {  
    if (allFinished(allProcesses)) break;  
  
    // check arrivals  
    for (Process p : allProcesses) {  
        if (p.arrival == time) {  
            enqueueByQueue(p, q0_SRT, q1_SJF, q2_RR, q3_FCFS);  
            System.out.println("Time " + time + ": Arrival -> " + p);  
        }  
    }  
}
```

-Then choosing the process to execute according to its priority

```
// اختيار العملية حسب الأولوية  
if (!q0_SRT.isEmpty()) {  
    running = getShortestRemaining(q0_SRT);  
    algo = "SRT";  
} else if (!q1_SJF.isEmpty()) {  
    running = getShortestJob(q1_SJF);  
    algo = "SJF";  
} else if (!q2_RR.isEmpty() || currentRR != null) {  
    if (currentRR == null) {  
        currentRR = q2_RR.remove(0);  
        rrCounter = 0;  
    }  
    running = currentRR;  
    algo = "RR";  
} else if (!q3_FCFS.isEmpty()) {  
    running = q3_FCFS.get(0); // peek  
    algo = "FCFS";  
} else {  
    break;  
}
```

- Context Switch happens every time a new process start running

```
// Context switch
if (lastRunning != running && lastRunning != null) {
    time++; // context switch
    gantt.add("CS");
    System.out.println("Time " + time + ": Context switch");
}
```

- As soon as the process is finished, its metrics are calculated.

```
if (running.remaining == 0) {
    running.finishTime = time + 1;
    running.turnaround = running.finishTime - running.arrival;
    running.waitingTime = running.turnaround - running.burst;
    System.out.println("Time " + (time + 1) + ": Process " + running.id + " finished. Turnaround=" +
        running.turnaround + ", Waiting=" + running.waitingTime + ", Response=" + running.responseTime);
    removeProcessFromAllQueues(running, q0_SRT, q1_SJF, q2_RR, q3_FCFS);
    if (running == currentRR) {
        currentRR = null;
        rrCounter = 0;
    }
}
```

- While the process is not running, its waiting time should be incremented.

```
static void incrementWaitingForQueues(List<Process> q0, List<Process> q1, List<Process> q2, List<Process> q3, Process running) {
    for (Process p : q0) if (p != running) p.waitingTime++;
    for (Process p : q1) if (p != running) p.waitingTime++;
    for (Process p : q2) if (p != running) p.waitingTime++;
    for (Process p : q3) if (p != running) p.waitingTime++;
}
```


Algorithms code

```
static Process getShortestRemaining(List<Process> q) {
    Process shortest = null;
    for (Process p : q) if (shortest == null || p.remaining < shortest.remaining) shortest = p;
    return shortest;
}
```

```
static Process getShortestJob(List<Process> q) {
    Process shortest = null;
    for (Process p : q) if (shortest == null || p.burst < shortest.burst) shortest = p;
    return shortest;
}
```

```
        if (running == currentRR) {
            currentRR = null;
            rrCounter = 0;
        }
    }
    //quantum :RR
    else if (algo.equals("RR")) {
        rrCounter++;
        if (rrCounter == quantum) {
            q2_RR.add(currentRR);
            currentRR = null;
            rrCounter = 0;
        }
    }
}
```

4.Test Cases output

```
public static void main(String[] args) {
    List<Process> processes = new ArrayList<>();

    processes.add(new Process("P1", 0, 6, 0)); // SRT
    processes.add(new Process("P2", 1, 8, 1)); // SJF
    processes.add(new Process("P3", 2, 7, 2)); // RR
    processes.add(new Process("P4", 3, 3, 3)); // FCFS
    processes.add(new Process("P5", 5, 2, 3));
    processes.add(new Process("P6", 6, 1, 2));

    simulateMLQ(processes, 2);
}
```

The output:

=== Simulation finished at time 33 ===

Per-process metrics:

ID	Arrival	Burst	Start	Finish	Waiting	Turnaround	Response
P1	0	6	0	6	0	6	0
P2	1	8	7	15	6	14	6
P3	2	7	16	26	17	24	14
P4	3	3	27	30	24	27	24
P5	5	2	31	33	26	28	26
P6	6	1	19	20	13	14	13

Average waiting = 14.33, Average turnaround = 18.83, Average response = 13.83

CPU Utilization = 81.82%, Throughput = 0.18 processes/unit time

Gantt Chart (time units):

| P1 | CS | P2 | CS | P3 | CS | P6 | CS | P3 | CS | P4 | CS | P5 |
0 6 7 15 16 18 19 20 21 26 27 30 31 33

5.Challenges Faces and it's Solution

Challenges

- **Process Starvation**: Low-priority queues can starve if high-priority processes constantly use the CPU.
- **Inflexibility**: Processes are fixed to their assigned queue, making it inefficient if their behavior changes (e.g., an interactive job becomes CPU-intensive)..
- **CPU Idleness**: High-priority queues might be empty, leading to CPU idle time while lower queues wait.

Solutions

- **Aging** Periodically increase the priority of processes in lower queues to prevent starvation and bring them into higher queues depending on a certain threshold for accumulating waiting
- **CPU IDLE**: When the highest priority queue finish its processes we move to another queue with lower priority to prevent waiting and all queues are empty we stop execution to only prevent an infinte loop of typing idle when there is no other process coming

Multilevel Feedback Queue

2.Algorithms Used

Theoretical Background



The idea is to give every new process a chance to run first, and if it keeps needing time, it goes down to other queues. It tries to balance fast response for small processes and still finish the long ones at the end.

It consists of multiple queues (usually RR & FCFS for the last queue) with different priority levels, Highest priority queue is RR with the least time quantum & Least priority queue is a FCFS

A new process always enters the highest priority queue (so all processes are executed for a time quanta)

If a process uses its entire time slice and still not finished, it is moved down to the next lower priority queue (Feedback Mechanism)

No lower priority queue is scheduled as long as higher one still has processes to be scheduled

Advantage and Disadvantage

Advantages:

- Flexibility & Responsiveness: Can handle mix of batch & interactive processes, ensuring all are served for a time quanta when entering at the first time
- Prevent starvation through aging: After a certain period, all processes are moved back to the highest priority queue

- **Good Throughput:** Use aging methods to ensure that long-running processes receive adequate CPU power. This balancing mechanism allows for the CPU to be fully utilized
- **Dynamic Priority Adjustment:** automatically adjusts the priority of a process based on its behavior. (Demotion & Boosting)
- **Efficient Use of CPU:** by preempting a process when its allocated time quantum expires.
- **Scalability:** handling a large number of processes simultaneously. It can effectively manage a high workload.

Disadvantages:

- **Complexity:** The most complex algorithm to implement & debug
- **System Overhead:** Moving processes between queues consume CPU cycles
- **Requires careful tuning** for parameters like number of queues, time slice size, boosting time.

Real Applications and Use case

Interactive Applications like Gaming & Multimedia Processing

3.Implementation Details Section

1- Process Class 2- MLFQSheduler Class 3- Main Class

In our project, we decided to use three queues with different priority levels.

The logic:

1. **Initialization:** Receive List of processes & initialize 3 queue: Q1 : Highest priority: RR 8 quantum, Q2: Medium priority: RR with 16 quantum, Q3: Least priority: FCFS
2. **Main Running Loop:** Ensure running until all processes are completed, check for every process if its arrival time has already come so that it can enter Q1, Choose the highest priority queue (not empty) to schedule its processes using the scheduling algorithm associated with that queue, if all are empty then CPU will be idle waiting for the next process

```
public static void run(List<Process> processes) {
```

```

int nextProcessIndex = 0;
while (completedProcesses.size() < allProcesses.size()) {

    while (nextProcessIndex < allProcesses.size() &&
           allProcesses.get(index: nextProcessIndex).arrivalTime <= currentTime) {
        q1.add(e: allProcesses.get(index: nextProcessIndex));
        nextProcessIndex++;
    }
    if (!q1.isEmpty()) {
        executeRR(currentQ: q1, timeQuantum: 8, nextQ: q2);
    } else if (!q2.isEmpty()) {
        executeRR(currentQ: q2, timeQuantum: 16, nextQ: q3);
    } else if (!q3.isEmpty()) {
        executeFCFS(currentQ: q3);
    } else {
        currentTime++; // Idle time
    }
}
}

```

3. Context Switch happens every time a new process will enter the CPU

```

private static void simulateContextSwitch() {
    if (totalBusyTime > 0 || !completedProcesses.isEmpty() || !q1.isEmpty() || !q2.isEmpty()
        || !q3.isEmpty()) {
        totalContextSwitches++;
        totalContextSwitchTime += CONTEXT_SWITCH_TIME;
        currentTime += CONTEXT_SWITCH_TIME;
    }
}

```

4. Scheduling Execution: if it is RR, then it gives the process a time slice of maximum value equal to the quanta or less if it has less remaining time or burst time. If it is FCFS, the process will be executed for its entire remaining time to be finished

5. Feedback: if the process still has remaining time, it is moved down to the lower priority queue to be served in it. For example, if process P1 runs for 8ms but still needs more time, we move it down to the second queue in the next cycle.

```

private void executeRR(Queue<Process> currentQ, int timeQuantum, Queue<Process> nextQ) {
    Process p = currentQ.poll();
    if (p.remainingTime < p.burstTime) {
        simulateContextSwitch();
    } else if (p.remainingTime == p.burstTime) {
        if (totalBusyTime > 0) {
            simulateContextSwitch();
        }
    }
    if (p.startTime == -1) p.startTime = currentTime;
    int timeSlice = Math.min(a: p.remainingTime, b: timeQuantum);
    totalBusyTime += timeSlice;
    currentTime += timeSlice;
    if (currentTime >= nextBoostTime) {
        performAging();
    }
    p.remainingTime -= timeSlice;
    if (p.remainingTime > 0) {
        nextQ.add(e: p);
    } else {
        finalizeProcess(p);
    }
}

```

```

private void executeFCFS(Queue<Process> currentQ) {
    Process p = currentQ.poll();
    if (p.remainingTime < p.burstTime) {
        simulateContextSwitch();
    } else if (p.remainingTime == p.burstTime) {
        if (totalBusyTime > 0) {
            simulateContextSwitch();
        }
    }
    if (p.startTime == -1) p.startTime = currentTime;
    int timeSlice = p.remainingTime;
    totalBusyTime += timeSlice;
    currentTime += timeSlice;
    if (currentTime >= nextBoostTime) {
        performAging();
    }
    p.remainingTime = 0;
    finalizeProcess(p);
}

```

6. Aging Mechanism: We added an aging mechanism. Basically, if a process stays too long in a low-priority queue, we push it back to the top queue. This prevents starvation and, in our tests, it helped slow processes get another chance to run.

```

private void performAging() {
    q1.addAll(c: q2);
    q2.clear();
    q1.addAll(c: q3);
    q3.clear();
    nextBoostTime += 50;
    System.out.println("Aging happened at time: " + currentTime);
}

```

4.Test Cases output

--- Process Execution Details ---

Multi Level Feedback Scheduler Output:

PID	Arrival	Burst	Priority	Start	Finish	TurnAround	Waiting
P1	0	6	1	0	6	6	0
P2	1	4	2	7	11	10	6
P3	2	7	3	12	19	17	10
P4	3	1	4	20	21	18	17
P5	5	4	4	22	26	21	17
P6	6	7	3	27	34	28	21

CPU Utilization: 85.29%

Total Context Switches: 5

Total Context Switch Time: 5

Average Waiting Time: 11.83

Average Turnaround Time: 16.67

Average Response Time: 11.83

Throughput: 0.1765 processes per time unit

Case with Aging:

Simulation Starting: MLFQ (Q1=8, Q2=16, Q3=FCFS)

Aging happened at time: 59

Aging happened at time: 109

PID	Arrival	Burst	Priority	Start	Finish	TurnAround	Waiting
1	0	40	1	0	59	59	19
2	2	4	2	9	13	11	7
3	4	10	3	14	42	38	28
4	80	30	1	81	116	36	6
5	82	2	2	90	92	10	8

CPU Utilization: 74.14%

Total Context Switches: 9

Total Context Switch Time: 9

Average Waiting Time: 13.60

Average Turnaround Time: 30.80

Average Response Time: 5.20

Throughput: 0.0431 processes per time unit

5.Challenges Faces and it's Solution

- Determining the process table that will simulate the feedback between all queues
- Implementing the Aging Mechanism
 - Solution: Tracking the Aging through a variable “nextBoostTime” that is compared to the “curentTime” variable to promote all processes if it exceeds the last one

Comparison of Each Algorithms by evaluation matrix

FINAL COMPARISON REPORT					
Algorithm	Avg Waiting	Avg Turnaround	Avg Response	CPU Util %	Throughput
FCFS	11.83	16.67	11.83	85.29 %	0.1765
SJF	9.83	14.67	9.83	85.29 %	0.1765
RR	15.33	20.17	9.67	78.38 %	0.1622
Priority Preemptive	15.33	20.17	13.50	80.56 %	0.1667
Priority Non-Preemptive	13.33	18.17	13.33	85.29 %	0.1765
SRT	11.00	15.83	8.17	80.56 %	0.1667
MLFQ	11.83	16.67	11.83	85.29 %	0.1765
MLQ	18.33	23.17	14.50	72.50 %	0.1500
WINNERS (Includes Ties):					
Best Avg Waiting Time : SJF (9.83)					
Best Avg Turnaround Time : SJF (14.67)					
Best Avg Response Time : SRT (8.17)					
Highest CPU Utilization : FCFS, SJF, Priority Non-Preemptive, MLFQ (85.29%)					
Highest Throughput : FCFS, SJF, Priority Non-Preemptive, MLFQ (0.1765)					

6. Conclusion

Process scheduling is a fundamental aspect of operating system design that directly impacts system performance and user experience. While simple algorithms like FCFS and Round Robin are easy to understand and implement, modern operating systems employ sophisticated multilevel feedback queue systems that adapt to different workload characteristics.

The choice of scheduling algorithm depends on system requirements, workload patterns, and performance objectives. Interactive systems prioritize response time and fairness, while batch systems focus on throughput and efficiency. Real-time systems require deterministic behavior and deadline guarantees.

7. References

- FCFS

[First Come First Serve CPU Scheduling- geeksforgeeks](#)

[FCFS Scheduling Algorithm-guru99](#)

- SJF

[Shortest Job First \(or SJF\) CPU Scheduling- geeksforgeeks](#)

[Shortest Job First \(SJF\): Preemptive, Non-Preemptive-guru99](#)

- SRT

[Shortest Remaining Time First \(Preemptive SJF\) Scheduling Algorithm - geeksforgeeks](#)

[Shortest remaining time - groklopedia](#)

[Difference between SJF and SRJF CPU scheduling algorithms - geeksforgeeks](#)

- RR

[Round Robin Scheduling in Operating System Geeks for Geeks](#)

[Round Robin Scheduling](#)

[Round Robin Scheduling Wikipedia](#)

-Priority algorithm

Priority-Scheduling-Algorithm

Priority Scheduling in Operating System Geeks for Geeks

Fixed-priority pre-emptive scheduling Wikipedia

- MLFQ

Multilevel Feedback Queue Scheduling (MLFQ) CPU Scheduling - GeeksforGeeks

Multilevel Feedback Queue Scheduling (MLFQ) CPU Scheduling-TutorialsPoint

- MLQ

Multilevel Queue (MLQ) CPU Scheduling - GeeksforGeeks

Multilevel queue Wikipedia